

Міністерство освіти і науки України
Національний Університет “Львівська Політехніка”



Лабораторна робота
з дисципліни
“Об'єктно-орієнтоване програмування, частина 2”

Виконав:
студент гр. ІХ-22
Ференц Роман

Прийняв:
Гордійчук-Бублівська О.В

Лабораторна робота 8

Тема: Реалізація клієнт-серверної взаємодії з використанням протоколу MQTT, автоматизація передачі даних через API.

Мета: Розробити клієнт-серверну архітектуру з використанням MQTT протоколу, реалізовувати класи для автоматизації роботи з API.

Завдання:

```
import requests
import asyncio
import websockets
import time
import paho.mqtt.client as mqtt

BROKER = "broker.hivemq.com"
PORT = 1883

class MQTTClient:

    def __init__(self):
        self.client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
        self.connected = False

        self.client.on_connect = self.on_connect
        self.client.on_disconnect = self.on_disconnect

    def on_connect(self, client, userdata, flags, reason_code, properties):
        if reason_code == 0:
            self.connected = True
            print("MQTT підключено до брокера")
        else:
            print("Помилка MQTT")

    def on_disconnect(self, client, userdata, disconnect_flags, reason_code, properties):
        print("MQTT відключено")

    def connect(self):
        self.client.connect(BROKER, PORT)
        self.client.loop_start()

    def client.loop_start()
    time.sleep(2)

    def publish(self, topic, message):
        self.client.publish(topic, message)
        print("MQTT опубліковано:", message)

    def disconnect(self):
        self.client.loop_stop()
        self.client.disconnect()

    def get_weather():
        url = "https://api.open-meteo.com/v1/forecast?latitude=49.84&longitude=24.03&current_weather=true"
        response = requests.get(url)
        data = response.json()
        temperature = data["current_weather"]["temperature"]
        wind = data["current_weather"]["windspeed"]
        print("REST API:")
        print("Температура:", temperature)
        print("Вітер:", wind)
        return temperature, wind

    async def send_websocket(message):
        uri = "wss://ws.ifelse.io"
        async with websockets.connect(uri) as websocket:
            await websocket.send(message)
            response = await websocket.recv()
            print("WebSocket отримав:")
            print(response)
```

```
async def main():
    mqtt_client = MQTTClient()
    print("Підключення MQTT...")
    mqtt_client.connect()
    temperature, wind = get_weather()
    message = f"Temperature: {temperature}, Wind: {wind}"
    print("Відправка через WebSocket...")
    await send_websocket(message)
    print("Публікація в MQTT...")
    mqtt_client.publish("lab5/weather", message)
    time.sleep(2)
    mqtt_client.disconnect()

await main()
```

Результат:

```
... Підключення MQTT...
MQTT підключено до брокера
REST API:
Температура: 12.5
Вітер: 5.8
Відправка через WebSocket...
WebSocket отримав:
Request served by 6e82931b755587
Публікація в MQTT...
MQTT опубліковано: Temperature: 12.5, Wind: 5.8
MQTT відключено
```

Висновок:

У ході виконання лабораторної роботи було розроблено клієнт-серверну архітектуру з використанням протоколу **MQTT** для обміну повідомленнями у реальному часі. Було створено клас **MQTTClient**, що дозволяє підключатися до брокера, публікувати повідомлення та відключатися.